



Cours d'algorithmique

Chapitre 1 : Complexité et algorithmes de références

**BDT & CIA & Sharkoders
Réseau Polytech
2025-2026**

I Crédits

Tout les droits sont réservés aux associations :

- **Bureau des technologies (BDT)** de Polytech Nice Sophia
- **Sharkoders** de Polytech Marseille
- **Le club d'informatique et d'algorithmique (CIA)** de Polytech Paris Saclay

Rédaction du cours :

- **Bilel MAHDJOUR**, Polytech Nice Sophia
- **Patrick BIZOT**, Polytech Nice Sophia

Relecture et correction :

- **Nino MULAC**, Polytech Nice Sophia

En tant qu'externe, vous êtes en droit de partager et d'utiliser le cours à but pédagogique, **seulement** lorsque vous avez obtenu l'autorisation auprès des associations et rédacteurs.

II Introduction

En informatique, nous sommes en général amenés à résoudre des problèmes très spécifiques. Par exemple, nous pouvons être à la recherche de la médiane d'un ensemble de valeurs, ou bien dans une situation plus difficile, être à la recherche de la plus grande sous-chaîne de caractères commune entre 2 chaînes de caractères. Pour ce faire, nous élaborons une série d'instructions pour permettre de répondre à ce problème, un **algorithme**.

Définition — Un **algorithme** est une suite d'étapes permettant d'obtenir un résultat à partir des données fournies en entrée.

Dans ce chapitre, nous ne nous attarderons pas à la terminaison d'un algorithme. Le but sera d'élaborer des notions pour comprendre comment étudier et élaborer un algorithme optimal.

II.1 Notations spécifiques

Ceci est une annexe, merci de vous y reporter lorsque vous avez un doute sur une notation.

Soit $A \subset \mathbb{N}$, X une variable aléatoire réelle. On suppose que $(A, \mathcal{P}(A), \mathbb{P})$ est un espace probabilisé où :

- $\mathbb{P}$ représente la fonction probabilité.
- $\mathcal{P}(A)$ représente l'ensemble des parties de A .
- $\mathbb{P}(X = d)$ représente la probabilité que X prenne la valeur d , avec $d \in A$.
- $\mathbb{E}$ représente la fonction qui renvoie l'espérance (moyenne pondérée) de X .

De même :

$$\sum_{d \in A}$$

indique que nous calculons la somme sur tout les éléments de A

Définition — Soient $(u_n), (v_n)$ des suites à valeurs dans $\mathbb{R}^+$, ne s'annulant pas à partir d'un certain rang.

Nous rappelons que (u_n) et (v_n) sont équivalentes au voisinage de $+\infty$, et on note $u_n \sim_{+\infty} v_n$, **si et seulement si** il existe une suite (w_n) tel que :

$$u_n = w_n v_n \text{ au voisinage de } +\infty \text{ et } \lim_{n \rightarrow +\infty} w_n = 1$$

III Complexité

Lorsque nous élaborons un algorithme, nous cherchons à ce qu'il soit optimal. Cette question d'optimalité rassemble plusieurs critères, cela peut être le temps ou encore l'espace mémoire utilisé. Comment allons-nous donc juger si notre algorithme est optimal ?

III.1 Notations asymptotiques

Pour étudier la rapidité d'un algorithme, nous allons étudier le temps pris par l'algorithme pour donner le résultat pour n données fournies. Ce nombre de données est étroitement lié à ce que manipule ce dernier, par exemple si on étudie un algorithme qui va trier une liste de valeurs, ce nombre n sera le nombre d'éléments dans cette liste.

Notons C , la fonction qui pour tout $n \in \mathbb{N}$, associe le temps pris par notre algorithme lorsqu'on lui fournis n données.

Pour voir à quel point notre algorithme est optimal, nous allons étudier son comportement asymptotique, c'est à dire, de quelle manière C va croître lorsqu'on augmente n .

Définition – La fonction C est un $\Omega(C')$ (dit « grand oméga de C' ») veut dire que :

$$\exists \alpha > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, C(n) \geq \alpha C'(n)$$

En langage naturel, cela revient à dire que $C(n)$ est **minorée** par $C'(n)$ à partir d'un certain rang, à une constante multiplicative près.

Définition – La fonction C est un $\mathcal{O}(C')$ (dit « grand O de C' » ou « dominée par C' ») veut dire que :

$$\exists \alpha > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, C(n) \leq \alpha C'(n)$$

En langage naturel, cela revient à dire que $C(n)$ est **majorée** par $C'(n)$ à partir d'un certain rang, à une constante multiplicative près.

Définition – La fonction C est $\Theta(C')$ (dit « du même ordre que C' ») veut dire que :

$$\exists \alpha, \beta \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, \alpha C'(n) \leq C(n) \leq \beta C'(n)$$

Remarques :

- $C = \Theta(C')$ si et seulement si $C = \Omega(C')$ et $C = \mathcal{O}(C')$.
- Au sens mathématique, soit $\alpha \in \mathbb{R}^*$, alors :

$$C \underset{n \rightarrow +\infty}{\sim} \alpha C' \implies C = \Theta(C').$$

- Il n'y a pas unicité des fonctions, je peux avoir $C = \Omega(Q)$ tout comme je peux avoir $C = \Omega(P)$, avec $P \neq Q$.

Exemple – Soit P la fonction de coût d'un algorithme, définie par :

$$\begin{aligned} P &: \mathbb{N} \rightarrow \mathbb{R} \\ | \quad n &\mapsto n^5 + 5n + 2 \end{aligned}$$

Nous avons par exemple que $P = \Omega(n^5)$ ou encore $P = \Omega(\log(n))$ à partir d'un certain rang. De même, $P = \mathcal{O}(n^5)$ ou encore $P = \mathcal{O}(n!)$. En particulier nous avons forcément $P = \Theta(n^5)$.

Par la suite, nous parlerons de **complexité d'un algorithme**, l'expression de la fonction de même ordre que la fonction de coût de l'algorithme. *Plus précisément, nous parlerons de complexité **temporelle**.*

III.2 Intérêts de la complexité

Les intérêts fondamentaux de la complexité temporelle sont de :

- Pouvoir comparer différents algorithmes entre eux, et donc sélectionner le plus optimal.
- Pouvoir évaluer et estimer le temps d'exécution d'un algorithme. Ceci pourra se faire en prenant en compte les spécificités du langage de programmation utilisé, et la fréquence de calcul du processeur.

III.3 Complexités de référence

Voici la liste des complexités de référence à absolument connaître pour établir la complexité d'un algorithme.

Nom	Complexité	Exemples d'utilisation
constante	$\Theta(1)$	Calcul d'aire d'un rectangle
logarithmique	$\Theta(\log(n))$	Recherche dichotomique dans une liste triée
linéaire	$\Theta(n)$	Recherche d'un éléments dans une liste

Nom	Complexité	Exemples d'utilisation
	$\Theta(n \log(n))$	Quicksort ou Tri fusion
quadratique	$\Theta(n^2)$	Tri par insertion ou par sélection
polynomiale	$\Theta(n^k)$	Recherche d'un élément dans un tableau à k dimensions
exponentielle	$\Theta(2^n)$	N-ième terme de la suite de Fibonacci récursivement sans DP
factorielle	$\Theta(n!)$	Problème du voyageur de commerce implémenté naïvement

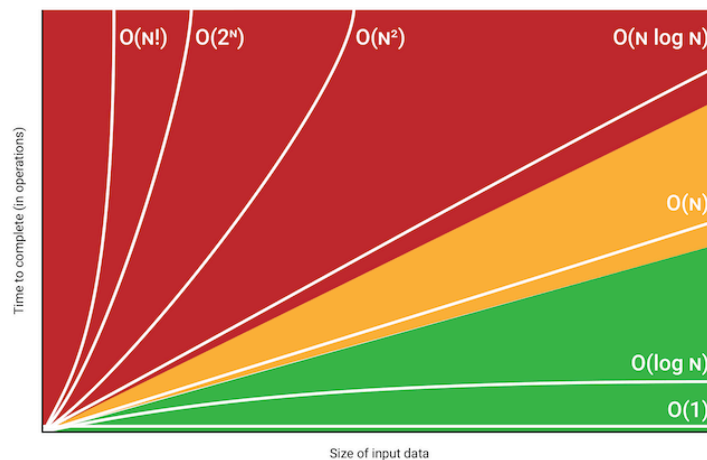


Fig. 1. – Comparaison des complexités de référence

III.4 Calculer la complexité d'un algorithme

Pour calculer la complexité d'un algorithme, en temps normal nous allons chercher à compter les instructions exécutées lorsque nous avons n données en entrée.

De manière générale, nous prenons en compte les tours de boucle et les comparaisons effectuées au sein de l'algorithme.

Mais **attention** ! Nous pouvons être piégés par cette technique, en particulier lorsque nous faisons des appels à des fonctions qui ont une complexité non constante ! C'est pour cela qu'il faut prendre en compte la complexité de l'instruction sélectionnée.

Il existe des cas de figure, où pour une donnée d'entrée de taille n , la complexité temporelle varie selon la représentation de la donnée.

C'est pour cela qu'il est utile de connaître la complexité dans les cas suivants :

- **Meilleur des cas** : La plus faible complexité qu'on puisse obtenir.
- **Pire des cas** : La plus grande complexité qu'on puisse obtenir.
- **En moyenne** : La complexité moyenne de l'algorithme.

Nous faisons une mention spéciale pour la **complexité en moyenne** qui n'est pas évidente à définir seulement avec son nom :

Définition — Soit X_n une variable aléatoire réelle, qui compte le nombre d'opérations effectuées selon la nature de la donnée de taille n — Nous noterons D_n , l'ensemble des données de taille n — alors la **complexité moyenne** est définie par :

$$T_{moy}(n) = \sum_{d \in D_n} \mathbb{P}(X_n = d) \times d$$

$$\text{ou } T_{moy}(n) = \mathbb{E}(X_n)$$

Exemple 1:

```
# Prime algorithm example
from math import sqrt

def is_prime(n : int) -> bool:

    if n < 2:
        return False

    for k in range(2, int(sqrt(n)) + 1):
        if n % k == 0:
            return False

    return True
```

Comme vous vous en doutez, l'algorithme précédent, écrit en Python, permet de déterminer si un entier naturel est premier ou non.

L'algorithme a :

- dans le meilleur des cas une complexité constante $\Omega(1)$ (Cas où n est pair).
- dans le pire des cas une complexité $\mathcal{O}(\sqrt{n})$. En effet nous obtenons cette dernière lorsque le nombre est premier, nous sommes donc contraint de voir pour $k \in \llbracket 1, \lfloor \sqrt{n} \rfloor \rrbracket$ si k divise n .
- en moyenne une complexité $\Theta\left(\frac{n}{\ln(n)}\right)$, la démonstration est laissée au lecteur.

Exemple 2:

```
# Max of a matrix algorithm example
from math import inf

type t_matrix = list[list[float]]
def max_matrix(matrix : t_matrix) -> float:

    N = len(matrix)
    max_val = -inf
    for i in range(N):
        for j in range(N):
            if matrix[i][j] > max_val:
                max_val = matrix[i][j]

    return max_val
```

Dans cet exemple, l'algorithme donné permet de déterminer le maximum des coefficients d'une matrice.

Cette fois-ci nous constatons que pour le meilleur et le pire des cas, la complexité est la même. Pourquoi ? Nous pouvons le voir très vite si nous nous concentrons sur les retours de valeurs. Dans notre exemple, il est impossible de faire un retour de valeur sans visiter chaque coefficients. Nous obtenons donc, après décompte des comparaisons faites, une complexité $\Theta(n^2)$.

III.5 Complexité spatiale

Définition — Dans la même idée que la complexité temporelle, nous définissons la **complexité spatiale**. Celle-ci décrit l'ordre de grandeur du nombre d'octets (ou bits) utilisés par rapport aux données d'entrée.

Cette notion est très importante en compétition, car le temps pris par un algorithme pour fonctionner n'est pas la seule contrainte. En effet, une limite de mémoire utilisée est aussi souvent imposée. En outre, les limites de mémoires peuvent être capitales dans d'autres cas, comme par exemple en programmation embarquée! Si vous souhaitez programmer sur une carte Arduino Uno, vous serez limités à 2ko de mémoire vive.

Exemple 3:

Nous supposons par la suite que l est une liste de valeurs entières et positives.

```
# Sort algorithm example
def bucket_sort(l : list[int]) -> list[int]:

    count = [0 for _ in range(max(l) + 1)]
    indices = [0 for _ in range(max(l) + 1)]
    temp = [None for _ in range(len(l))]

    for element in l:
        count[element] += 1

    for i in range(1, len(indices)):
        indices[i] = indices[i - 1] + count[i - 1]

    for element in l:
        index = indices[element]
        temp[index] = element
        indices[element] += 1

    return temp
```

Dans cet algorithme, comme pour le dernier exemple, nous avons une complexité temporelle similaire entre le pire et meilleur des cas, qui sera $\Theta(n)$. Cependant sa complexité spatiale sera de $\Theta(\max(n, \max(l)))$ si on étudie les variables définies (en l'occurrence nous définissons des liste de valeurs). Cela rend l'usage de cet algorithme assez limité, malgré l'avantage considérable de performance, comparé à la plupart des algorithmes de tri.

IV Algorithmes de référence

Les algorithmes traités dans cette section seront revus plus en détail dans l'année, dans leur propre chapitre.

IV.1 Recherche par dichotomie

La recherche par **dichotomie** est un algorithme permettant d'obtenir l'emplacement d'un élément dans une liste d'éléments triée. L'idée est simple, supposons qu'on cherche l'élément E :

- On va lire l'élément au milieu de la liste qu'on nomme V .
- Supposons que les éléments de la liste sont triés selon une relation d'ordre $\leq$, alors on va distinguer 3 cas :
 - Si $E = V$, alors c'est gagné, nous avons l'emplacement de l'élément E .
 - Si $E < V$, alors on divise par 2 la zone de recherche, et on garde la partie **strictement à gauche** de V , et on revient à l'étape précédente.
 - Si $E > V$, alors on divise par 2 la zone de recherche, et on garde la partie **strictement à droite** de V , et on revient à l'étape précédente.

Voici un exemple de recherche dichotomique, faite sur une liste d'entiers triée, pour savoir si un nombre est présent dans la liste :

```
# Dichotomy research
def is_present(l : list, x : int) -> bool:

    left = -1
    right = len(l)
    middle = None

    while left + 1 < right:
        middle = (left + right) // 2
        if l[middle] == x:
            return True
        elif l[middle] < x:
            left = middle
        else:
            right = middle

    return False
```

La recherche dichotomique est très importante, car elle permet de passer d'une complexité moyenne en $\Theta(n)$ pour une simple recherche dans une liste, à une complexité moyenne en $\Theta(\log n)$, ce qui asymptotiquement, fait une grosse différence.

La recherche par dichotomie est généralisable pour tout ensemble de valeurs admettant certaines caractéristiques, mais nous resterons sur cette base pour ce chapitre.

IV.2 Algorithmes de tri

Pour trier des listes, il existe une multitude d'algorithmes. Nous y retrouvons entre autres des algorithmes performants comme le quicksort et le tri fusion (complexité en $\Theta(n \log n)$). Nous n'aborderons pas ces méthodes de tri ici, mais plutôt des méthodes classiques, qui ont certes une complexité en $\Theta(n^2)$, mais qui ont la particularité d'être camouflées dans les exercices, et d'être facile à comprendre.

IV.2.A Tri par sélection

Le tri par sélection consiste à :

- Récupérer l'élément le plus petit élément d'une liste.
- Le déplacer au début de la liste.

Et on continue en mettant le second élément le plus petit, après le premier, jusqu'à obtenir une liste triée.

Voici une implémentation en Python de l'algorithme :

```
# Selection sort
def sort(l : list[int]):
    min_i = 0
    for i in range(len(l)):
        min_i = i
        for j in range(i, len(l)):
            if l[j] < l[min_i]:
                min_i = j
        temp = l[i]
        l[i] = l[min_i]
        l[min_i] = temp
```

Complexité	Meilleur des cas	Pire des cas	Moyenne
Temporelle	$\Omega(n^2)$	$\mathcal{O}(n^2)$	$\Theta(n^2)$
Spatiale	$\Omega(1)$	$\mathcal{O}(1)$	$\Theta(1)$

On remarque que la complexité temporelle ne varie pas selon les cas, car l'algorithme ne s'arrête pas quand la liste est déjà triée.

IV.2.B Tri par insertion

L'idée de l'algorithme est de considérer 2 parties dans la liste :

- Une partie triée (gauche)
- Une partie non triée (droite)

Et à chaque fois on prend le premier élément de la partie droite, et on l'insère à la bonne place dans la partie gauche.

Voici une implémentation en Python de l'algorithme :

```
# Insertion sort
def sort(l : list[int]):
    for i in range(1, len(l)):
        el = l[i]
        j = i
        while j > 0 and l[j - 1] > el:
            l[j] = l[j - 1] # décalage de l'élément à l'indice j - 1 vers la droite
            j -= 1
        l[j] = el
```

Complexité	Meilleur des cas	Pire des cas	Moyenne
Temporelle	$\Omega(n)$	$\mathcal{O}(n^2)$	$\Theta(n^2)$
Spatiale	$\Omega(1)$	$\mathcal{O}(1)$	$\Theta(1)$

Remarques :

- Le meilleur des cas est atteint lorsque la liste est déjà triée.
- Le pire des cas est atteint lorsque la liste est triée dans l'ordre inverse.